How does an operating system implement file protection, and what are the common access control techniques used?

File protection in an operating system refers to the methods used to control how files and directories are accessed, especially in environments where multiple users share the system. It ensures that only authorized users can perform permitted operations on files, preventing misuse, accidental changes, or malicious actions.

One core aspect of file protection is managing types of access rights, which include:

Read – Viewing the content of a file.
Write – Modifying the contents.
Execute – Running a file as a program.
Append – Adding data to the end of a file.
Delete – Removing the file.
List – Viewing the file name and its attributes (especially for directories).
Higher-level operations like copying or renaming are combinations of these fundamental rights. The degree and nature of protection required often depend on the system type—for example, academic, personal, or enterprise-level systems.

A common technique for enforcing protection is through Access Control Lists (ACLs). These are lists attached to each file, specifying which users (or groups) are allowed to perform which operations. When a user attempts to access a file, the operating system checks the ACL to determine whether the requested action is permitted. If not, access is denied. This offers a flexible and user-specific method of access control.

Another technique is password-based protection, where access to a file or directory is granted only when the correct password is provided. While simple, this method has drawbacks:

Users may have to remember many passwords.
A shared password across multiple files creates a single point of failure—if it's compromised, all associated files are at risk.
To address these issues, systems may:

Use passwords at the directory level rather than per-file.
Require different passwords for different actions (e.g., one for read access, another for write access).
These techniques help maintain system security by regulating file access and minimizing the risk of unauthorized operations.

Question:
What is free space management in file systems, and what are the different techniques used to implement it?

Answer:
Free space management in file systems refers to the process of tracking and allocating unused disk blocks so they can be assigned to files as needed.
The main techniques used are:

- 1. Bit Vector (Bitmap)

- Uses a bit array where 0 = free, 1 = allocated.

Diagram:
Block No: 0 1 2 3 4 5 6 7
Bitmap: 1 0 0 1 1 0 0 1

- 2. Linked List

- Free blocks are linked together as a list.

Diagram:
[3] -> [7] -> [10] -> [12] -> NULL

- 3. Grouping

- First free block contains addresses of others and pointer to next group.

Diagram:
[Block 5] → [6, 8, 9, 12, Block 15]
[Block 15] → [18, 20, 21, NULL]

- 4. Counting

- Stores the starting block and count of contiguous free blocks.

Diagram:
[Start: 5, Count: 4] → Blocks 5, 6, 7, 8
[Start: 12, Count: 3] → Blocks 12, 13, 14

Question:
What are file system calls in operating systems, and what are the commonly used file system calls with their purposes?

Answer:
File system calls are system-level functions that allow programs to perform operations like creating, reading, writing, and deleting files. These calls act as an interface between the user program and the operating system's file system.

Commonly Used File System Calls

open()
Purpose: Opens a file for reading, writing, or both.
Syntax:int fd = open("file.txt", O_RDONLY);

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main() {
int fd = open("example.txt", O_RDONLY);
if (fd < 0) {
perror("open");
return 1;
}
printf("File opened with fd: %d\n", fd);
close(fd);
return 0;
}
```

read()
Purpose: Reads data from a file into a buffer.
Syntax: ssize_t n = read(fd, buffer, size);

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main() {
char buf[100];
```

```
int fd = open("example.txt", O_RDONLY);
if (fd < 0) {
perror("open");
return 1;
}
ssize_t n = read(fd, buf, sizeof(buf)-1);
if (n > 0) {
buf[n] = '\0';
printf("Read: %s\n", buf);
}
close(fd);
return 0;
}
```

write()
Purpose: Writes data from a buffer to a file.
Syntax: ssize_t n = write(fd, buffer, size);

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
#include <string.h>

int main() {
int fd = open("output.txt", O_WRONLY | O_CREAT, 0644);
if (fd < 0) {
perror("open");
return 1;
}
const char *msg = "Hello, File System!";
write(fd, msg, strlen(msg));
close(fd);
return 0;
}
```

close()
Purpose: Closes an open file descriptor.
Syntax: int status = close(fd);

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main() {
int fd = open("example.txt", O_RDONLY);
if (fd < 0) {
perror("open");
return 1;
}
close(fd);
printf("File closed.\n");
return 0;
}
```

creat()
Purpose: Creates a new file or rewrites an existing one.
Syntax:int fd = creat("newfile.txt", 0644);

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main() {
int fd = creat("newfile.txt", 0644);
if (fd < 0) {
perror("creat");
return 1;
}
printf("File created with fd: %d\n", fd);
close(fd);
return 0;
}
```

unlink()
Purpose: Deletes a file from the file system.
Syntax: int status = unlink("file.txt");

```
#include <unistd.h>
#include <stdio.h>

int main() {
if (unlink("newfile.txt") == 0) {
printf("File deleted successfully.\n");
} else {
perror("unlink");
}
return 0;
}
```

lseek()
Purpose: Moves the file pointer to a specified position.
Syntax: off_t pos = lseek(fd, offset, SEEK_SET);

SEEK_SET — Set pointer to a specific offset from start

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main() {
int fd = open("example.txt", O_RDONLY);
if (fd < 0) {
perror("open");
return 1;
}
lseek(fd, 5, SEEK_SET); // Move to 5th byte from start
char ch;
read(fd, &ch, 1);
printf("Character at offset 5: %c\n", ch);
close(fd);
return 0;
}
```

SEEK_CUR — Move relative to current position

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main() {
int fd = open("example.txt", O_RDONLY);
if (fd < 0) {
perror("open");
return 1;
}
lseek(fd, 2, SEEK_CUR); // Skip 2 bytes ahead of current position
char ch;
read(fd, &ch, 1);
printf("Character after skipping 2 bytes: %c\n", ch);
close(fd);
return 0;
}
```

SEEK_END — Move relative to end of file

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main() {
int fd = open("example.txt", O_RDONLY);
if (fd < 0) {
perror("open");
return 1;
}
off_t size = lseek(fd, 0, SEEK_END);
printf("File size: %ld bytes\n", size);
close(fd);
return 0;
}
```

stat()
Purpose: Retrieves information about a file.
Syntax: int status = stat("file.txt", &statbuf);

```
#include <sys/stat.h>
#include <stdio.h>

int main() {
struct stat statbuf;
if (stat("example.txt", &statbuf) == 0) {
printf("File size: %ld bytes\n", statbuf.st_size);
printf("Last modified: %ld\n", statbuf.st_mtime);
} else {
perror("stat");
}
return 0;
}
```